MidEng GK8.2 Document Oriented Middleware using MongoDB [GK/EK] - 4h

@author: David Socaciu

@version:2026-04-21

Als erstes habe ich alles vorbereitet für die Übung indem ich auf Docker mit `docker pull mongo` mir den Docker Container gepulled habe und auch das Github-Repository gekloned habe.

Zuerst habe ich dann eine Datei `application.properties` erstellt habe und die wichtigsten Werte gesetzt habe:

```
spring.data.mongodb.host=localhost
spring.data.mongodb.port=27017
spring.data.mongodb.database=warehouseDB
```

Anschließend an dem habe ich dann eine neue Klasse `Warehouse` erstellt damit ich auch mehrere Lagerstandorte haben kann mit folgenden Attributen und dazu gehörigen Setter & Getter Methoden:

```
    @Id
    private String id;

    private String warehouseID;
    private String warehouseName;
    private String warehouseCity;

    private String timestamp;
    private int warehousePostalCode;
    private String warehouseCountry;

    private List<ProductData> productData;
```

Da ich die ID im Warehouse jetzt habe muss ich eine kleinere Änderung in der gegebenen Klasse `ProductData` vornehmen:

Und zwar die ID rauslöschen/auskommentieren da die ID in Warehouse ist.

Auch das Repository musste ich etwas anpassen und daher sieht die Kopfzeile jetzt wie folgt aus:

```
public interface WarehouseRepository extends MongoRepository<Warehouse, String> {
```

Ich habe auch hier einfach das `ProductData` durch `Warehouse` ersetzt.

Nun fehlt mir noch der Controller der folgende Kriterien erfüllen muss:

- POST /warehouse: fügt einen neuen Lagerstandort hinzu.
- GET /warehouse: abrufen aller Lagerstandorte und deren Lagerbestand
- GET /warehouse/{id}: abrufen eines Lagerstandortes id und dessen Lagerbestand
- DELETE /warehouse/{id}: löschen eines Lagerstandortes id
- POST /product: fügt ein neues Produkt und dessen Lagerbestand zu einem Lagerstandort hinzu
- GET /product: abrufen aller Produkte/Lagerbestand und deren Lagerstandort
- GET /product/{id}: abrufen eines Produktes id und dessen Lagerstandorte
- DELETE /product/{id}: löschen eines Produktes id auf einem Lagerstandort

So sehen die Methoden aus fürs Warehouse:

```java
    @PostMapping("/warehouse")
    public Warehouse addWarehouse(@RequestBody Warehouse warehouse) {
        return repository.save(warehouse);
    }

    // GET /warehouse
    @GetMapping("/warehouse")
    public List<Warehouse> getAllWarehouses() {
        return repository.findAll();
    }

    // GET /warehouse/{id}
    @GetMapping("/warehouse/{id}")
    public Warehouse getWarehouseByID(@PathVariable String id) {
        return repository.findByWarehouseID(id);
    }

    // DELETE /warehouse/{id}
    @DeleteMapping("/warehouse/{id}")
    public String deleteWarehouse(@PathVariable String id) {
        Warehouse warehouse = repository.findByWarehouseID(id);

        if (warehouse != null) {
            repository.delete(warehouse);
            return "Warehouse deleted: " + id;
        } else {
            return "Warehouse not found: " + id;
        }
    }
```

Wobei hier alle Methoden das repository verwenden für die jeweiligen CRUD-Operationen.

Bei den Produkten sieht es ähnlich aus:

```
    // POST /product/{warehouseID}
    @PostMapping("/product/{warehouseID}")
    public Warehouse addProduct(@PathVariable String warehouseID, @RequestBody
ProductData product) {

        Warehouse warehouse = repository.findByWarehouseID(warehouseID);

        if (warehouse != null) {
            warehouse.getProductData().add(product);
            return repository.save(warehouse);
        }

        return null;
    }

    // GET /product (alle Produkte aus allen Warehouses)
    @GetMapping("/product")
    public List<Warehouse> getAllProducts() {
        return repository.findAll();
    }
```

Abschließend habe ich die Klasse `Application.java` geändert, sodass wie in der Angabe verlangt mindestens 10 Produkte in 3 Produktkategorien erstellt werden wobei ich das auf zwei Warehouses geteilt habe

So sieht zb ein Auschnitt vom ersten Warehouse aus

```
Warehouse w1 = new Warehouse(
                "1",
                "Linz Bahnhof",
                "Linz",
                "2022-01-02 01:00:00",
                4010,
                "Austria",
                Arrays.asList(
                        new ProductData("1", ....),
                        )
        );
```

Und dann kann ich über die das Rest-Template die REST-API aufrufen und hinzufügen

```
        restTemplate.postForObject(
                "http://localhost:8080/api/warehouse",
                w1,
                Warehouse.class
        );
```

Nun kann ich überprüfen ob die Inhalte in der Datenbank gespeichert wurden und man sieht, dass alles passend in der DB ist.

```
warehouseDB> db.warehouse.find().pretty()
[
  {
    _id: ObjectId('69e759bb2c3bcc2d3b0e8d75'),
    warehouseID: '1',
    warehouseName: 'Linz Bahnhof',
    warehouseCity: 'Linz',
    timestamp: '2022-01-02 01:00:00',
    warehousePostalCode: 4010,
    warehouseCountry: 'Austria',
    productData: [
      {
        warehouseID: '1',
        productID: '00-443175',
        productName: 'Bio Orangensaft Sonne',
        productCategory: 'Getraenk',
        productQuantity: 2500
      },
      {
        warehouseID: '1',
        productID: '00-871895',
```

Auch über die API sieht man das es funktioniert:

localhost:8080/api/warehouse

uelltextformatierung


```
{
  "id": "69e759bb2c3bcc2d3b0e8d75",
  "warehouseID": "1",
  "warehouseName": "Linz Bahnhof",
  "warehouseCity": "Linz",
  "timestamp": "2022-01-02 01:00:00",
  "warehousePostalCode": 4010,
  "warehouseCountry": "Austria",
  "productData": [
    {
      "warehouseID": "1",
      "productID": "00-443175",
      "productName": "Bio Orangensaft Sonne",
      "productCategory": "Getraenk",
      "productQuantity": 2500
    },
    {
      "warehouseID": "1",
      "productID": "00-871895",
      "productName": "Bio Apfelsaft Gold",
      "productCategory": "Getraenk",
      "productQuantity": 3420
    },
    {
      "warehouseID": "1",
      "productID": "01-926885",
      "productName": "Ariel Waschmittel Color",
      "productCategory": "Waschmittel",
      "productQuantity": 478
    },
    {
      "warehouseID": "1",
      "productID": "04-111111",
      "productName": "Coca Cola 1L",
      "productCategory": "Getraenk",
      "productQuantity": 1500
    },
    {
      "warehouseID": "1",
      "productID": "05-222222",
      "productName": "Persil Waschmittel",
      "productCategory": "Waschmittel",
      "productQuantity": 900
    }
  ]
```


Auch über Postman habe ich einen POST-Request getestet:

POST http://localhost:8080/api/warehouse Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
{
  "warehouseID": "3",
  "warehouseName": "Graz Lager",
  "warehouseCity": "Graz",
  "timestamp": "2022-01-05 12:00:00",
  "warehousePostalCode": 8010,
  "warehouseCountry": "Austria",
  "productData": []
}
```

Body Cookies Headers (5) Test Results 200 OK 150 ms 374 B

{} JSON Preview Visualize

```
{
    "id": "69e762692c3bcc2d3b0e8d77",
    "warehouseID": "3",
    "warehouseName": "Graz Lager",
    "warehouseCity": "Graz",
    "timestamp": "2022-01-05 12:00:00",
    "warehousePostalCode": 8010,
    "warehouseCountry": "Austria",
    "productData": []
}
```

Dann habe ich noch folgendes gemacht:

5 CRUD Operationen über Mongo Shell Dokumentieren Sie den Mongo Shell Befehl und dessen Ergebnis. Beispiel: ein Produkt hinzufügen, ein Produkt löschen, ein Produkt ändern, ...

Produkt hinzufügen:

```
db.warehouse.updateOne(
  { warehouseID: "1" },
  {
    $push: {
      productData: {
        warehouseID: "1",
        productID: "99-999999",
        productName: "Neues Produkt",
        productCategory: "Test",
        productQuantity: 500
      }
    }
  }
)
```

Produkt suchen

Damit kann ich zb das soeben erstellte Produkt suchen.

```
db.warehouse.find(
  { "productData.productID": "99-999999" }
).pretty()
```

Und dann kommt das Warehouse welches das Produkt enthält

```
      warehouseID: '1',
      productID: '99-999999',
      productName: 'Neues Produkt',
      productCategory: 'Test',
      productQuantity: 500
    }
  ],
```

Produkt ändern

Nun änder ich das Produkt bzw. die Quantity davon:

```
db.warehouse.updateOne(
  { "productData.productID": "99-999999" },
  {
    $set: {
      "productData.$.productQuantity": 1
    }
  }
)
```

Und auch das hat funktioniert

Produkt löschen

Nun lösche ich das Produkt zum Abschluss noch:

```
db.warehouse.updateOne(
  { warehouseID: "1" },
  {
    $pull: {
      productData: { productID: "99-999999" }
    }
  }
)
```

Und auch das Produkt wurde gelöscht und wenn ich danach suche sieht man das es nicht mehr existiert:

```
warehouseDB> db.warehouse.find(
|    { "productData.productID": "99-999999" }
| ).pretty()

warehouseDB>
```

Fragen

- **Nennen Sie 4 Vorteile eines NoSQL Repository im Gegensatz zu einem relationalen DBMS** Flexibles Schema (kein fixes Tabellenlayout)

  Gute Skalierbarkeit Schnelle Verarbeitung großer Datenmengen Einfach für JSON /dokumentbasierte Daten

- **Nennen Sie 4 Nachteile eines NoSQL Repository im Gegensatz zu einem relationalen DBMS**

Weniger Konsistenz Keine Komplexen JOINS Teilweise schlechtere Standardisierung Daten können redundater sein

- **Welche Schwierigkeiten ergeben sich bei der Zusammenführung der Daten? Unterschiedliche Datenformate/Strukturen** Dateninkonsistenz möglich Duplikate entstehen leicht Keine JOINs → schwieriger zu kombinieren

- **Welche Arten von NoSQL Datenbanken gibt es?**

  Key-Value Dokumentenorientiert Spaltenorientiert Graphdatenbanken

- **Nennen Sie einen Vertreter für jede Art?** Key-Value → Redis Dokumentenorientiert → MongoDB Spaltenorientiert → Cassandra Graph → Neo4j

- **Beschreiben Sie die Abkürzungen CA, CP und AP in Bezug auf das CAP Theorem CA** → Consistency + Availability (kein Partitionstoleranz) **CP** → Consistency + Partition Tolerance **AP** → Availability + Partition Tolerance

- **Mit welchem Befehl koennen Sie den Lagerstand eines Produktes aller Lagerstandorte anzeigen.**

  ```
  db.warehouse.find(
    { "productData.productID": "00-443175" },
    { "productData.$": 1 }
  )
  ```

- **Mit welchem Befehl koennen Sie den Lagerstand eines Produktes eines bestimmten Lagerstandortes anzeigen.**

```
db.warehouse.find(
  { warehouseID: "1", "productData.productID": "00-443175" },
  { "productData.$": 1 }
)
```

Vertiefung

Generieren Testdaten für das Berichtswesen: mind. 300 Produkte in 6 Produktkategorien, 5 Warenhaeuser

So habe ich das umgesetzt:

```java
        String[] categories = {
                "Getraenk","Waschmittel","Tierfutter",
                "Reinigung","Snacks","Hygiene"
        };

        for (int w = 1; w <= 5; w++) {

            List<ProductData> products = new ArrayList<>();

            for (int i = 1; i <= 60; i++) {
                products.add(new ProductData(
                        String.valueOf(w),
                        "P" + w + i,
                        "Produkt " + i,
                        categories[i % 6],
                        (int)(Math.random() * 1000)
                ));
            }

            Warehouse warehouse = new Warehouse(
                    String.valueOf(w),
                    "Warehouse " + w,
                    "City " + w,
                    "2022",
                    1000 + w,
                    "Austria",
                    products
            );

            restTemplate.postForObject(
                    "http://localhost:8080/api/warehouse",
                    warehouse,
                    Warehouse.class
            );
```

Ich habe zuerst sechs Kategorien erstellt und in einem Array gespeichert.

Dann habe ich ne FOR-Schleife gemacht für 5 Warehouses und dann noch eine wo ich dann jeweils 60 Produkte erstelle und speichere diese in eine Liste und füg sie dann auch zum Warehouse hinzu und abschließend füge ich dann das gesamte warehouse in die DB hinzu.

Formulierung 3 sinnvoller Fragestellungen für das Berichtswesen in der Zentrale und deren Abfragen in einer Mongo Shell. Beispiel: Wie ist der Lagerbestand von einem Produkt X über alle Lagerstandorte? Welche Produkte haben einen Lagerbestand von unter 10 Stück über alle Lagerstandorte?

Frage 1

Wie hoch ist der gesamte Lagerbestand eines Produkts (z.B. Coca Cola) über alle Lagerstandorte?

```
db.warehouse.aggregate([
  { $unwind: "$productData" },
  { $match: { "productData.productID": "04-111111" } },
  { $group: {
      _id: "$productData.productID",
      gesamt: { $sum: "$productData.productQuantity" }
  }}
])
```

```
warehouseDB> db.warehouse.aggregate([
|    { $unwind: "$productData" },
|    { $match: { "productData.productID": "04-111111" } },
|    { $group: {
|        _id: "$productData.productID",
|        gesamt: { $sum: "$productData.productQuantity" }
|    }}
| ])
[ { _id: '04-111111', gesamt: 2500 } ]
```

Frage 2

Welche Produktkategorie hat über alle Lagerstandorte den höchsten Gesamtbestand?

```
db.warehouse.aggregate([
  { $unwind: "$productData" },
  { $group: {
      _id: "$productData.productCategory",
      gesamt: { $sum: "$productData.productQuantity" }
  }},
  { $sort: { gesamt: -1 } },
  { $limit: 1 }
])
```

```
warehouseDB> db.warehouse.aggregate([
|    { $unwind: "$productData" },
|    { $group: {
|        _id: "$productData.productCategory",
|        gesamt: { $sum: "$productData.productQuantity" }
|    }},
|    { $sort: { gesamt: -1 } },
|    { $limit: 1 }
| ])
[ { _id: 'Reinigung', gesamt: 38323 } ]
warehouseDB>
```

Frage 3

Wie viele verschiedene Produkte liegen in jedem Warehouse

```
db.warehouse.aggregate([
  { $unwind: "$productData" },
  { $group: {
      _id: "$warehouseID",
      anzahlProdukte: { $sum: 1 }
  }},
  { $sort: { _id: 1 } }
])
```

```
arehouseDB> db.warehouse.aggregate([
   { $unwind: "$productData" },
   { $group: {
       _id: "$warehouseID",
       anzahlProdukte: { $sum: 1 }
   }},
   { $sort: { _id: 1 } }
 ])

 { _id: '1', anzahlProdukte: 60 },
 { _id: '10', anzahlProdukte: 5 },
 { _id: '2', anzahlProdukte: 60 },
 { _id: '20', anzahlProdukte: 7 },
 { _id: '3', anzahlProdukte: 60 },
 { _id: '4', anzahlProdukte: 60 },
 { _id: '5', anzahlProdukte: 60 }

arehouseDB>
```